

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №7-8
По дисциплине: «Естественно-языковой интерфейс ИС»
Тема: «Разработка систем анализа речи», «Разработка систем синтеза речи»

Выполнил:
Студент 4 курса
Группы ИИ-23
Романюк А. П.

Проверил:
Булей Е. В.

Брест 2025

Цель: освоить на практике основные принципы создания систем анализа и синтеза речи.

9	Синтеза	Английский,	Критика предметов изобразительного искусства
---	---------	-------------	--

Ход работы:

- ☐ Ввод текста, или копирование через буфер, или поддержка указателя мыши в любом другом приложении (например, на html-страничке).
- ☐ Воспроизведение сгенерированного речевого сигнала для введенного текста.
- ☐ Настройки (выбор) голоса диктора, темпа, громкости и др. параметров для чтения
- ☐ описание структуры разработанной системы, технологий и методик;
- ☐ основные алгоритмы и принципы реализации компонент системы (блок-схемы алгоритмов);
- ☐ результаты тестирования;
- ☐ результаты анализа полученных данных, и предложения по улучшению работы системы.

Задание 1:

```
import tkinter as tk
from tkinter import ttk
import speech_recognition as sr
import threading

class SpeechApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Приложение распознавания речи")

        self.label = tk.Label(root, text="Нажмите кнопку и говорите:", font=("Arial", 16))
        self.label.pack(pady=20)

        self.result_text = tk.Text(root, height=10, width=50)
        self.result_text.pack(pady=20)

        self.language_var = tk.StringVar(value='ru-RU') # По умолчанию русский
        self.language_label = tk.Label(root, text="Выберите язык:", font=("Arial", 12))
        self.language_label.pack(pady=5)

        self.language_combobox = ttk.Combobox(root, textvariable=self.language_var, values=[
            'ru-RU', 'en-US', 'de-DE', 'fr-FR'])
        self.language_combobox.pack(pady=5)

        self.recognize_button = tk.Button(root, text="Распознать речь", command=self.start_recognition_thread, font=("Arial", 12))
        self.recognize_button.pack(pady=20)

    def start_recognition_thread(self):
        threading.Thread(target=self.recognize_speech).start()

    def recognize_speech(self):
        recognizer = sr.Recognizer()
        recognizer.pause_threshold = 1
        with sr.Microphone() as source:
            self.label.config(text="Слушаю...")
            #self.root.update() # Обновляем интерфейс, чтобы метка изменилась
            #recognizer.adjust_for_ambient_noise(source, duration=1)
        try:
            audio = recognizer.listen(source, timeout=3, phrase_time_limit=12)
        except sr.WaitTimeoutError as e:
            self.label.config(text=f"Не удалось получить результаты; {e}.")
        try:
            language = self.language_var.get()
            text = recognizer.recognize_google(audio, language=language)
            print(text)
            self.result_text.delete(1.0, tk.END)
```

```

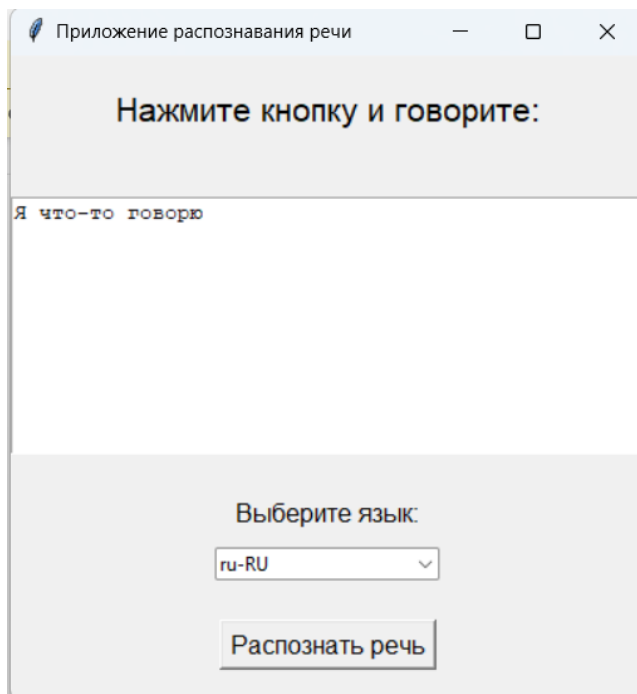
self.result_text.insert(tk.END, text)
self.label.config(text="Нажмите кнопку и говорите:")

self.respond_to_command(text)

except sr.UnknownValueError:
    self.label.config(text="Не удалось распознать аудио.")
except sr.RequestError as e:
    self.label.config(text=f"Не удалось получить результаты; {e}.")
except sr.WaitTimeoutError as e:
    self.label.config(text=f"Не удалось получить результаты; {e}.")
def respond_to_command(self, text):
    if "привет" in text.lower():
        self.label.config(text="Привет! Как я могу помочь?")
    elif "стоп" in text.lower():
        self.label.config(text="А я что машина?")
    elif "как дела" in text.lower():
        self.label.config(text="У меня все хорошо, спасибо!")

if __name__ == '__main__':
    root = tk.Tk()
    app = SpeechApp(root)
    root.mainloop()

```



Задание 2:

```

import tkinter as tk
from tkinter import ttk
import pyttsx3

class TTSApp:
    def __init__(self, root):
        self.root = root
        self.root.title("🔊 Система синтеза речи")
        self.root.geometry("600x550")
        self.root.resizable(False, False)

        style = ttk.Style()
        style.theme_use("clam")
        style.configure("TButton", font=("Arial", 12), padding=6)
        style.configure("TLabel", font=("Arial", 12))
        style.configure("TCombobox", font=("Arial", 11))
        style.configure("TLabelframe.Label", font=("Arial", 13, "bold"))

```

```

engine = pyttsx3.init()
self.voices = engine.getProperty("voices")
self.voice_map = {v.name: v.id for v in self.voices}

text_frame = ttk.LabelFrame(root, text=" Текст для озвучивания ")
text_frame.pack(fill="both", padx=15, pady=10)

self.text_entry = tk.Text(text_frame, height=8, wrap="word", font=("Arial", 12))
self.text_entry.pack(fill="both", padx=10, pady=10)

settings_frame = ttk.LabelFrame(root, text=" Настройки ")
settings_frame.pack(fill="x", padx=15, pady=10)

ttk.Label(settings_frame, text="Голос:").grid(row=0, column=0, padx=5, pady=5, sticky="w")
self.voice_var = tk.StringVar(value=list(self.voice_map.keys())[0])
self.voice_combobox = ttk.Combobox(settings_frame, textvariable=self.voice_var,
                                   values=list(self.voice_map.keys()), width=40, state="readonly")
self.voice_combobox.grid(row=0, column=1, padx=5, pady=5, sticky="ew")

ttk.Label(settings_frame, text="Скорость:").grid(row=1, column=0, padx=5, pady=5, sticky="w")
self.speed_scale = tk.Scale(settings_frame, from_=50, to=300, orient="horizontal", length=300)
self.speed_scale.set(150)
self.speed_scale.grid(row=1, column=1, padx=5, pady=5)

ttk.Label(settings_frame, text="Громкость:").grid(row=2, column=0, padx=5, pady=5, sticky="w")
self.volume_scale = tk.Scale(settings_frame, from_=0.0, to=1.0, resolution=0.1,
                             orient="horizontal", length=300)
self.volume_scale.set(1.0)
self.volume_scale.grid(row=2, column=1, padx=5, pady=5)

settings_frame.columnconfigure(1, weight=1)

buttons_frame = ttk.Frame(root)
buttons_frame.pack(pady=20)

self.speak_button = ttk.Button(buttons_frame, text="► ПРОИЗНЕСТИ", command=self.speak_text)
self.speak_button.grid(row=0, column=0, padx=10)

def speak_text(self):
    text = self.text_entry.get("1.0", tk.END).strip()
    if not text:
        return
    self.speak(text)

def speak(self, text):
    engine = pyttsx3.init()

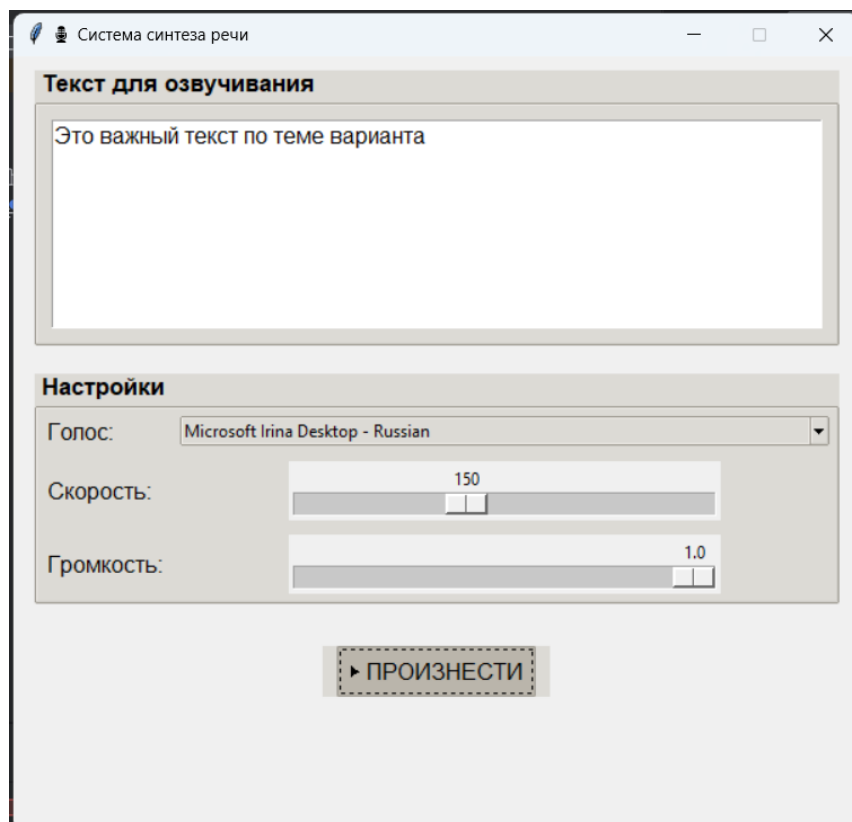
    selected_name = self.voice_var.get()
    voice_id = self.voice_map.get(selected_name, list(self.voice_map.values())[0])

    engine.setProperty("voice", voice_id)
    engine.setProperty("rate", self.speed_scale.get())
    engine.setProperty("volume", self.volume_scale.get())

    engine.say(text)
    engine.runAndWait()

if __name__ == "__main__":
    root = tk.Tk()
    app = TTSAApp(root)
    root.mainloop()

```



Вывод: в ходе выполнения лабораторной работы освоил на практике основные принципы создания систем анализа и синтеза речи.